ZARC

ZARC CP/M USERS' GUIDE

Written by Merlin Skinner-Oakes, 21/5/2025

Revision	*Date*	*Changes*
1	13/5/2022	Created
2	21/5/2025	Modified to include CP/M 3

1 DESCRIPTION

The Z80 Anachronistic Retro Computer (ZARC) supports CP/M 2.2 and CP/M 3 (also known as CP/M Plus).

CP/M 2.2 is normally started from the monitor programme using the following command:

```
> cpm2
```

Similarly, CP/M 3 is normally started from the monitor programme using the following command:

```
> cpm3
```

This document describes the customised CP/M BIOS, memory card format and ZARC-specific utilities. Information on CP/M itself is widely available online.

In this document, the following style conventions are used:

```
Machine output
User commands
Comments
```

2 CP/M 2.2

This section describes features specific to CP/M 2.2. The monitor's cpm2 command is used to start the boot process. This command may optionally be followed by one or more of the following options separated by spaces:

- debug – display additional debugging messages, notably one for each BIOS call.
- noints – use polled serial I/O instead of interrupts.
- rawcon – disable console terminal translation.

Example monitor command line:

```
> cpm2 debug
```

At the end of each CP/M session, run the command:

```
> zarc exit
```

This ensures a controlled exit including flushing the memory card cache.

2.1 Boot Process

The monitor programme implements the boot process. This is started by the user entering the following command:

```
> cpm2
ZARC CP/M V2.2 BIOS V1.6

A>
```

The overall flow is as follows:

1) Load the master boot record (MBR) from sector 0.
2) Verify boot signature is present (0x55 0xaa at 0x01be).
3) Scan the partition table, looking for an "experimental" partition, which ZARC uses for CP/M. *$_1$
4) Allocate four memory pages "CPM0" to "CPM3" to provide a 64 kB space.
5) Load the BIOS from the start of the partition.
6) Verify that it is a ZARC BIOS.
7) Execute it.

*$_1$ no official Z80 / 8080 CP/M partition type is coded, so this seems the most sensible choice for this application.

2.2 Memory Map

While CP/M 2.2 is active, memory is mapped as follows:

```
0 (0000 - 3FFF): 00 [CPM0]   writeable
1 (4000 - 7FFF): 01 [CPM1]   writeable
2 (8000 - BFFF): 02 [CPM2]   writeable
3 (C000 - FFFF): 03 [CPM3]   writeable, supervisor
```

Note that bank 3 is the only one with I/O (supervisor) access. This contains the BIOS, and is the one bank maintained when calls are made to the monitor. For this reason, interrupt code is maintained in this bank. Page numbers for CP/M may vary to avoid clashes with other applications, and are allocated using the monitor's MPALL function.

When the monitor is active for debugging or when its support routines are called, the map is modified:

```
0 (0000 - 3FFF): 3f [MONP]   read only, supervisor
1 (4000 - 7FFF): 3e [MOND]   writeable
2 (8000 - BFFF): (not used)
3 (C000 - FFFF): 03 [CPM3]   writeable
```

Pages are mapped into bank 2 (8000 - BFFF) as required to allow access to the full CP/M address space.

3 CP/M 3

CP/M 3 is also known as CP/M Plus. The version used for ZARC is actually CP/M 3.1, but is referred to in this document (and others, including those from DRI) as CP/M 3.

This section describes features specific to CP/M 3. The monitor's cpm3 command is used to start the boot process. This command may optionally be followed by one or more of the following options separated by spaces:

debug – display additional debugging messages, notably one for each BIOS call.

rawcon – disable console terminal translation.

Example monitor command line:

```
> cpm3 debug
```

At the end of each CP/M session, run the command:

```
> zarc exit
```

This ensures a controlled exit including flushing the memory card cache.

3.1 PROFILE.SUB

To have commands run on startup, insert them in A:PROFILE.SUB. Examples:

```
SETDEF *,A:
SETDEF [UK]
```

The first command sets the search order for programmes. The second sets UK date formatting.

3.2 Boot Process

The monitor programme implements the boot process. This is started by the user entering the following command:

```
> cpm3
ZARC CPMLDR BIOS V1.0 for CP/M 3
CP/M V3.0 Loader
Copyright (C) 1998, Caldera Inc.

 BNKBIOS3 SPR  EE00  0E00
 BNKBIOS3 SPR  A900  1700
 RESBDOS3 SPR  E800  0600
 BNKBDOS3 SPR  7B00  2E00

 58K TPA
ZARC CP/M 3 BIOS V1.0, build: Mon 19/08/24 13:37:02
A>
```

The overall flow is as follows:

1) Load the master boot record (MBR) from sector 0.
2) Verify boot signature is present (0x55 0xaa at 0x01be).
3) Scan the partition table, looking for an "experimental" partition, which ZARC uses for CP/M. *$_1$
4) Allocate thirteen memory pages "CPM0" to "CPMC" to provide four 48 kB CP/M banks and a 16 kB common area.
5) Load DRI's CPMLDR from the system area of the partition. This contains a minimal implementation of CP/M Plus and a BIOS.
6) Verify that it appears to be executable code.
7) Execute it.
8) CPMLDR loads CPM3.SYS and executes it.
9) CPM3.SYS initializes the system and then loads CCP.COM
10) The CCP displays the command prompt.

*$_1$ no official Z80 / 8080 CP/M partition type is coded, so this seems the most sensible choice for this application. Note that this partition is shared with CP/M 2.2.

Since the main parts of the system are files (CPM3.SYS and CCP.COM), the system may be updated simply by copying updated versions of these files to the boot disk.

3.3 Memory Map

The word “bank” has two meanings in this context. For ZARC generally, it means the four 16 KB areas of the Z80’s 64 KB addressable memory space that may each be mapped to any one of the RAM pages. In CP/M 3, bank means the non-common area that may be mapped to any one of a number of RAM pages. For ZARC, the common area is 0xc000 to 0xffff (ZARC bank 3) and is always mapped to the same RAM page. ZARC banks 0 to 2 (0x0000 to 0xbfff) are mapped as one unit to any of the available CP/M banks. I shall refer to “ZARC bank” or “CP/M bank” when discussing CP/M 3 to make it clear which definition is intended.

Four CP/M banks are allocated. This is somewhat excessive, but ZARC has lots of RAM and in any case this arrangement may make it easier to run MP/M as a future project. From the point of view of the monitor and CPMLDR.COM, memory is initialised as for CP/M 2.2, i.e. CP/M bank 0 is active (CP/M 3 System Guide section 1.9).

While CP/M 3 is active, memory is mapped as follows:

0x0000 - 0xbfff: currently selected CP/M bank (see table below).

0xc000 – 0xffff: common area, always page “CPMC” regardless of CP/M bank.

CP/M Bank	***ZARC Page at 0x0000 - 0x3fff***	***ZARC Page at 0x4000 – 0x7fff***	***ZARC Page at 0x8000 - 0xbfff***
0	CPM0	CPM1	CPM2
1	CPM3	CPM4	CPM5
2	CPM6	CPM7	CPM8
3	CPM9	CPMA	CPMB

Note that the common area (ZARC bank 3) is the only one with I/O (supervisor) access. This contains the BIOS, and is the one ZARC bank maintained regardless of CP/M bank and when calls are made to the monitor. For this reason, interrupt code is maintained in this area. Page numbers for CP/M may vary to avoid clashes with other applications, and are allocated using the monitor's MPALL function.

When the monitor is active for debugging or when its support routines are called, the map is modified:

```
0 (0000 - 3FFF): 3f [MONP]   read only, supervisor
1 (4000 - 7FFF): 3e [MOND]   writeable
2 (8000 - BFFF): (not used)
3 (C000 - FFFF): 03 [CPMC]   writeable
```

Pages are mapped into ZARC bank 2 (8000 - BFFF) as required to allow access to the full CP/M address space.

3.3.1 Load Map

On boot, CP/M 3 displays a load map, for example:

```
BNKBIOS3 SPR  EF00  0D00   Common section of BIOS (cseg)
BNKBIOS3 SPR  AD00  1300   Bank 0 section of BIOS (dseg)
RESBDOS3 SPR  E900  0600   Common section of BDOS (cseg)
BNKBDOS3 SPR  7F00  2E00   Bank 0 section of BDOS (dseg)
```

The common bank has BDOS at E900 followed directly by the BIOS at EF00. E900 is 58.25 KB, hence the 58 K TPA report from CPMLDR. test_bios.sym may be used to find the location of modules within each section of the BIOS for debugging.

3.4 Year 2000 Fixes

From https://forum.vcfed.org/index.php?threads/installing-cp-m-3-plus-on-a-home-built-z80-computer.61802/page-22

John Elliot writes:

“When I did the Year 2000 fixes, I also added support for multiple date formats. Use SETDEF to select: SETDEF [US], SETDEF [UK] or SETDEF [YMD]."

“durgadas311”: "I found that the date format is being stored in SCB byte at offset 0CH (12). Value 00 for "US", 01 for "UK", and 02 for "YMD". This byte is part of a "reserved for system use" section, but does not appear to be used by the system otherwise."

http://www.cpm.z80.de/binary.html

cpm3bin.zip contains year 2000 fixes.

4 CP/M BIOS

Features common to both CP/M 2.2 and 3.1 BIOS are described initially, followed by sections that describe features unique to one or the other.

4.1 Extended Functions

In addition to the conventional BIOS functions, there are some additional ones. These are defined in cpm.i. Applications must check for the presence of the ZARC BIOS before using extended functions. The function ZRCDET in cpm_utility.z80 is a convenient method.

Jump Table Entry	*Function*
BIOS_ID	"ZRC"
BIOS_VER	<reserved>, <major>, <minor>
STARTUP	ZARC CP/M startup entry point
MON_CALL	Call monitor function
SET_SUPER	Set supervisor state
INT_DISABLE	Disable interrupt and keep count
INT_ENABLE	Enable interrupts if safe.
TIRD	Read time and date (ZARC format)
SET_TIMER	Set elapsed time (centiseconds)
GET_TIMER	Get elapsed timer (centiseconds)

The following sub-sections provide an overview of each function. For full details such as register usage, see the source file bios.z80.

4.1.1 BIOS_ID

The first extended entry is simply the three-byte ASCII string “ZRC”. This identifies the BIOS are intended for ZARC, and may be used as a rationalness check before relying on ZARC-only functions. Three bytes are used to maintain alignment with the other jump table entries.

4.1.2 BIOS_VER

The version is reported as follows:

Byte	*Function*
0	Reserved (zero). May be used to flag capabilities in the future.
1	Major version
2	Minor version

Three bytes are used to maintain alignment with the other jump table entries.

4.1.3 STARTUP

startup is an entry point to the BIOS, and is used to pass initial parameters such as memory mapping information from the monitor programme to the BIOS. See source files bios/bios.z80 and its counterpart in the monitor that calls it (monitor/cpm.z80) for details.

4.1.4 MON_CALL

This routine allows CP/M applications to call some monitor functions. Any function that accesses memory must be used with caution as ZARC banks 0 and 1 will be changed to select the monitor's environment.

4.1.5 SET_SUPER

Conventional CP/M programmes exclusively use the BIOS to access I/O. However, it is possible to write code that directly accesses the I/O space. Without supervisor mode, any attempt to access I/O from ZARC banks 0 to 2 (0x0000 to 0xbfff) will result in a NMI and subsequent return to the monitor. In order to circumvent this, CP/M programmes should call this function to enable or disable supervisor access for ZARC banks 0 to 2 as required. Bank 3 (0xc000 to 0xffff) is left unchanged as the BIOS always requires supervisor mode in order to function.

Call with:

A - non-zero to enable supervisor mode, zero to disable.

4.1.6 INT_DISABLE and INT_ENABLE

Nested interrupt enable / disable pairs present an issue in that the interrupts are enabled after the first enable is encountered, which is likely unsafe. Use of int_disable and int_enable avoid this as interrupts are only enabled when the outermost enable is executed. This is achieved by maintaining a record of the disable count.

4.1.7 TIRD

Reads time and date as a single 32-bit integer. This is similar to the monitor's routine of the same name, but this version reads the interrupt maintained time instead. The system keeps track of time as a Unix-style 32-bit number. Unlike Unix, this is treated as an unsigned integer and so will not suffer from the year 2038 problem as it doubles the allowable positive time. It does, however, mean that it is not possible to represent dates before 1970. Leap seconds are ignored. 32-bit time does not include BST adjustment and so always represents GMT. However, the monitor’s tiitos (convert integer time to structure) and tistoi (convert structure time to integer) add or remove the appropriate offset and so describe "clock time" without further adjustment.

Returns with:

A - Tick divider (0 to TICK_FREQ -1)

DEHL - 32-bit time

4.1.8 SET_TIMER

Set elapsed time as a 32-bit integer. This is incremented on each tick interrupt to maintain elapsed time in centiseconds.

Call with:

BCDE - 32-bit time in centiseconds (0 to approximately 497 days).

4.1.9 GET_TIMER

Read elapsed time as a 32-bit integer.

Returns with:

BCDE - 32-bit time in centiseconds (0 to approximately 497 days).

4.2 Console Terminal Translation

Console input and output sequences are translated to provide a simple interface to CP/M applications while using a widely available ANSI / VT-100 terminal or emulator externally. A useful guide to control strings is to be found at https://vt100.net/docs/vt510-rm/chapter4.html

Translation may be disabled by starting CP/M from the monitor with the "rawcon" option. Translation of console input depends on interrupts, so is disabled if CP/M is started with the "noints" option.

The translation is identical between CP/M 2.2 and CP/M 3 and is performed by the module contran.z80. This file and contran.i are identical for both CP/M versions. The is one subtle difference, however. For CP/M 2.2, all console device data is translated, regardless of the physical device to which it is mapped. CP/M 3 has a more sophisticated mapping system between logical and physical devices. It allows multiple consoles. It is assumed that ZARC SER1 is connected to the only console and data is translated to and from that port. Data associated with other devices is not translated regardless of which device is mapped as console.

4.2.1 Console Output

Translations performed are:

ZARC Sequence	*VT-100 / ANSI Sequence*	*Function*	*Comment*
Esc A (1B 41)	Esc [A	Cursor up	Move cursor one line upwards.
Esc B (1B 42)	Esc [B	Cursor down	Move cursor one line downwards.
Esc C (1B 43)	Esc [C	Cursor right	Move cursor one column to the right.
Esc D (1B 44)	Esc [D	Cursor left	Move cursor one column to the left.
Esc H (1B 48)	Esc [H	Cursor home	Move cursor to the upper left corner.
Esc J (1B 4A)	Esc [J	Clear to end of display	Clear screen from cursor onwards.
Esc K (1B 4B)	Esc [K	Clear to end of line	Clear line from cursor onwards.
Esc Y <y> <x> (1B 59 YY XX)	Esc [<y> ; <x> H	Set cursor position	Move cursor to position <x - 32>, <y - 32>, encoded as single characters. Origin (top left-hand corner) is (0, 0).
Esc p (1B 70)	Esc [7 m	Reverse video on	
Esc q (1B 71)	Esc [27 m	Reverse video off	
Esc r (1B 72)	Esc [4 m	Underline on	
Esc u (1B 75)	Esc [24 m	Underline off	
Esc L (1B 4C)	Esc [L	Insert line	Insert line at cursor, pushing lines down
Esc M (1B 4D)	Esc [M	Delete line	Delete line at cursor, pulling lines up
Esc E (1B 45)	Esc [H Esc [J	Home cursor and clear screen	
Esc d (1B 64)	Esc [1 J	Clear to start of display	
Esc e (1B 65)	Esc [25 h	Enable cursor	
Esc f (1B 66)	Esc [25 l	Disable cursor	

4.2.2 Console Input

Input escape codes are translated to single characters where appropriate. This is the approach taken by later CP/M machines such as the Amstrad PCW range and Spectrum +3. Translations performed are:

Key	*Sequence*	*ZARC Code*	*Origin*
Cursor up	Esc [A	0x1f (^_)	Amstrad PCW and Spectrum +3
Cursor down	Esc [B	0x1e (^^)	Amstrad PCW and Spectrum +3
Cursor right	Esc [C	0x06 (^F)	Amstrad PCW and Spectrum +3
Cursor left	Esc [D	0x01 (^A)	Amstrad PCW and Spectrum +3
Home	Esc [1 ~ or Esc [H or Esc O H	0x02 (^B)	Unique to ZARC
Insert	Esc [2 ~	0x16 (^V)	Unique to ZARC
Delete	Esc [3 ~	0x7f	Common
End	Esc [4 ~ or Esc [F or Esc O F	0x04 (^D)	Unique to ZARC
Page up	Esc [5 ~	0x1d (^])	Unique to ZARC
Page down	Esc [6 ~	0x1c (^\)	Unique to ZARC

Unrecognised codes are passed on untranslated, so it is possible for software to communicate with other terminal types without disabling the translation.

4.2.3 Rationale

VT-52 codes were chosen for output as it offers relatively simple control codes and is the basis for a variety of later CP/M machines such as the Amstrad PCW and Spectrum +3.

Few terminal emulators support ADM-3A, and some CP/M programmes cannot support VT-100 sequences as these are relatively complex to generate. Some software such as BBC Basic cannot generate these codes as the space allocated to customisation is too small.

ADM-3A codes are initially attractive as earlier Kaypro machines mimic this. Later ones (“2.2G” and later) use different cursor control codes. These include ^S, so would interfere with XON / XOFF flow control.

Console input is translated to single character codes for ease of customisation. This appears to be what the Spectrum +3 and Amstrad PCW range used.

4.3 CP/M 2.2 BIOS

This section describes features unique to the CP/M 2.2 BIOS.

This is based on details in Digital Research CP/M Operating System Manual, especially appendix A, "The MDS Basic I/O System (BIOS)" and appendix B, "A Skeletal CBIOS", both available at:

http://www.gaby.de/cpm/manuals/archive/cpm22htm/.

5 MEMORY CARD FORMAT

The space used by the CP/M system is located in a single 24 MB partition partition on the card. This is shared between the CP/M 2.2 and CP/M 3 environments. It allows for 23 1 MB disks with an extra (very generous) 1 MB for CP/M 2.2's BDOS and CCP. It is also used for CP/M 3's CPMLDR.COM.

Note that one large disk would be hard to organise given there are no sub-directories in CP/M.

This section describes the format of data on the memory card. CP/M 3 shares the same partition with CP/M 2.2. The partition space is allocated as follows:

Offset from Partition Start (MB)	**Size (MB)**	**CP/M 2.2 Function**	**CP/M 3 Function**
0	1	System space	System space
1	1	A:	Not accessible
2	7	B: to H: $*_2$	B: to H: $*_2$
9	1	P: $*_1$	A: $*_1$
10	14	Spare space (not accessible)	Spare space (not accessible)

Notes:

$*_1$ This is one shared drive visible to both systems. This allows CP/M 2.2 to write system files using which CP/M 3 can boot.

$*_2$ These drives are accessible to both systems.

Though CP/M 2.2 can readily handle more drives, CP/M 3 has a limitation in this regard as each drive requires buffer space. This must be in bank 0. The choice of 8 drives allows 7 directory and 18 data buffers per disk.

5.1 Partition Table

The partition table may be loaded and examined using the monitor.

```
Setup memory mapping.
> map 0 0                    Map page 0 to bank 0
> set mmap 1                 Ensure memory mapping is enabled
> readsec 0 1 0000           Read the boot sector
> dump 0000 0200
0000  FA B8 00 10 8E D0 BC 00  |........|
```

```
0008  B0 B8 00 00 8E D8 8E C0  |........|
0010  FB BE 00 7C BF 00 06 B9  |...|....|
0018  00 02 F3 A4 EA 21 06 00  |.....!..|
0020  00 BE BE 07 38 04 75 0B  |....8.u.|
0028  83 C6 10 81 FE FE 07 75  |.......u|
0030  F3 EB 16 B4 02 B0 01 BB  |........|
0038  00 7C B2 80 8A 74 01 8B  |.|...t..|
0040  4C 02 CD 13 EA 00 7C 00  |L.....|.|
0048  00 EB FE 00 00 00 00 00  |........|
0050  00 00 00 00 00 00 00 00  |........|
                ...
01B8  06 97 85 77 00 00 00 04  |...w....|
01C0  01 04 7F 63 02 64 00 08  |...c.d..|
01C8  00 00 00 C0 00 00 00 00  |........|
01D0  00 00 00 00 00 00 00 00  |........|
                ...
01F8  00 00 00 00 00 00 55 AA  |......U.|
```

01fe is the boot signature (0x55 0xaa).

0x01BE is the start of partition entry no. 1

Partition type (0x7f) is manually edited by me (readsec, modify, then writesec).

Red - LBA of first absolute sector in the partition (0x00000800 [2048d] or 1024 KB [1 MB])

Green - number of sectors in partition (0x00008800 [49152d] sectors or 24576 KB [24 MB])

Hence start sector for the CP/M partition is 0x0800 (2048d).

5.2 CP/M Disk Parameter Block (DPB)

This CP/M structure defines the disk geometry. For ZARC, all disks are identical.

Data	*CP/M Name*	*Description*
0100	SPT	sectors per track
04	BSH	data allocation block shift factor
0F	BLM	data allocation block mask (2^[BSH-1])
00	EXM	extent mask
01FF	DSM	maximum block number
01FF	DRM	maximum directory entry number
FF, 00	AL0, AL1	determine reserved directory blocks
0000	CKS	size of the directory check vector
Varies *$_1$	OFF	number of reserved tracks

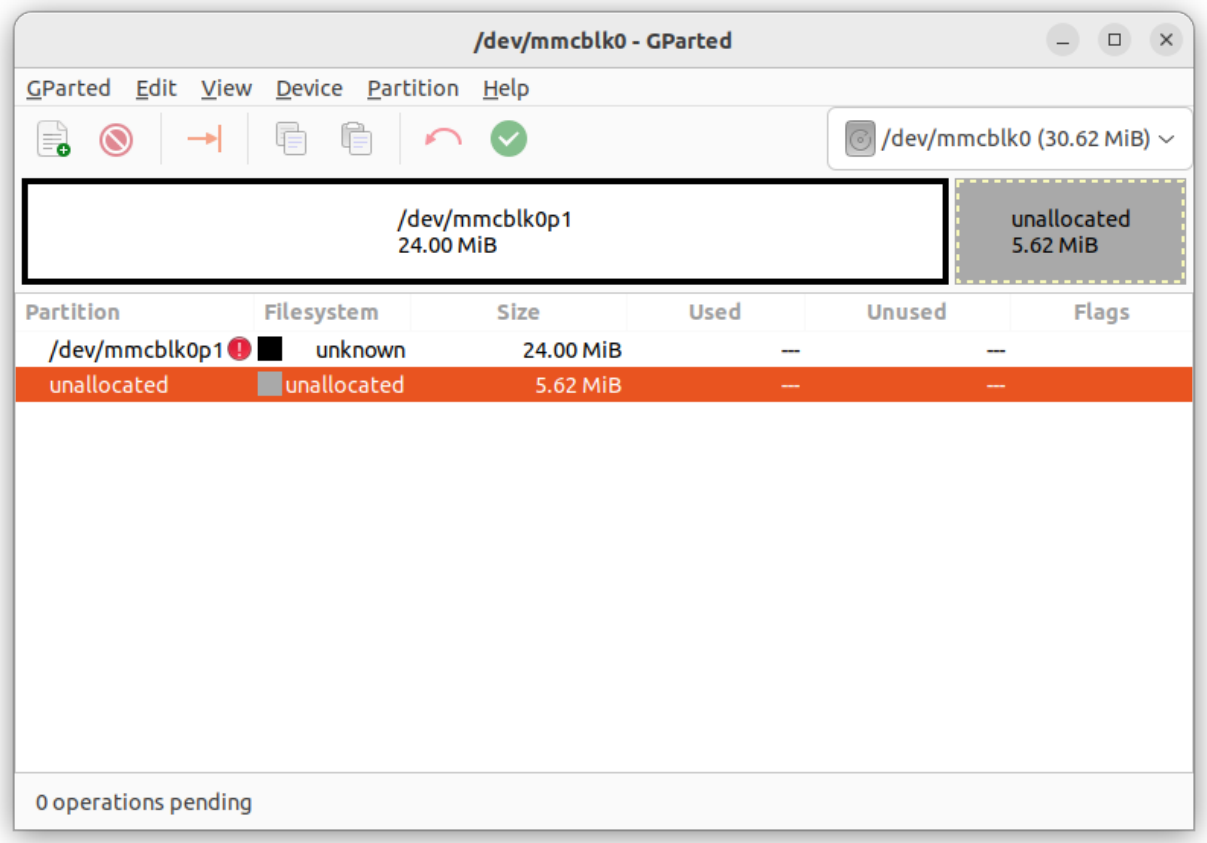
*$_1$ For CP/M 2.2, OFF is different for each disk. It is used to define the start of the disk within the partition. For CP/M 3, the offset is handled in the disk driver.

6 UPDATES AND INSTALLATION

This section describes how to initialise an memory card and bring up the CP/M system.

6.1 Creating the Partition

The required partition was created using GParted on a Linux system.



Unfortunately, it seems common, if not universal, on PC systems to start the first partition 1 MB from the start of the disk. Manual editing could overcome this, but it doesn't seem worthwhile, in part due to the complications of the CHS addressing system.

GParted cannot create the correct partition type, so this was changed using the monitor:

```
Setup memory mapping.
> map 0 0                    Map page 0 to ZARC bank 0
> set mmap 1                 Ensure memory mapping is enabled
> readsec 0 1 0000           Read the boot sector
> write 01c2 7f
> dump 01C0 10
01C0  01 04 7F 63 02 64 00 08  |...c.d..|
01C8  00 00 00 C0 00 00 00 00  |........|
> writesec 0 1 0000          Write the modified boot sector
```

6.2 Installing the System Data

The CP/M 2.2 BIOS, BDOS and CCP plus CP/M 3 CPMLDR all must be written to the correct locations in the partition.

The following steps achieve this. Note that commands entered at the "$" prompt are to be issued from the Linux system.

The components of CP/M are downloaded using a script called zarc_dl. This overcomes the limitations of the USB serial adaptor used to connect the PC to ZARC's console port.

Note that it is necessary to have a working CP/M 2.2 system before CP/M 3 can be installed.

6.2.1 Install CP/M 2.2 BIOS

If modifications to the CP/M 2.2 BIOS are required, edit the source files in the cpm/bios folder and then run "make" to produce bios.cim.

Follow these steps to install or update the BIOS:

```
Setup memory mapping.
> map 3 3                    Map page 3 to ZARC bank 3
> set mmap 1                 Ensure memory mapping is enabled
> bindownload ec00           Download to BIOS start
$ zarc_dl bios.cim /dev/ttyUSB0
[reset]
> writesec 2059 8 ec00       Write to memory card
```

Be careful not to send any extraneous characters when using bindownload. There is no header, length or checksum so any data sent will be written to memory.

6.2.2 Install CP/M 2.2 BDOS and CCP

ccp.bin is 2048 bytes (16 CP/M sectors) – loaded to 0xd600

bdos.bin is 3584 bytes (28 CP/M sectors) – loaded to 0xde00

This is 5632 bytes (0x1600) or 11 MMC sectors in total.

Follow these steps to install or update the BDOS and CCP.

```
Setup memory mapping.
> map 3 3                    Map page 3 to ZARC bank 3
> set mmap 1                 Ensure memory mapping is enabled
> bindownload d600           Download to CCP start
The BDOS immediately follows the CCP, so we can transfer it in
one tranfer.
$ zarc_dl ccp.bin /dev/ttyUSB0
$ zarc_dl bdos.bin /dev/ttyUSB0
[reset]
> dump d600                  Display start of CCP
D600  C3 5C D9 C3 58 D9 7F 00  |.\..X...|
D608  20 20 20 20 20 20 20 20  |        |
```

```
 ...
D618  43 4F 50 59 52 49 47 48  |COPYRIGH|
D620  54 20 28 43 29 20 31 39  |T (C) 19|
D628  37 39 2C 20 44 49 47 49  |79, DIGI|
D630  54 41 4C 20 52 45 53 45  |TAL RESE|
D638  41 52 43 48 20 20 00 00  |ARCH  ..|
> dump de00                    Display start of BDOS
DE00  00 00 00 00 00 00 C3 11  |........|
DE08  DE 99 DE A5 DE AB DE B1  |........|
DE10  DE EB 22 43 E1 EB 7B 32  |.."C..{2|
DE18  D6 EB 21 00 00 22 45 E1  |..!.."E.|
DE20  39 22 0F E1 31 41 E1 AF  |9"..1A..|
DE28  32 E0 EB 32 DE EB 21 74  |2..2..!t|
DE30  EB E5 79 FE 29 D0 4B 21  |..y.).K!|
DE38  47 DE 5F 16 00 19 19 5E  |G._....^|
> writesec 2048 11 d600      Write CCP and BDOS to memory card
```

6.2.3 Install CPMLDR

CPMLDR is required for CP/M 3 and is allocated 2560 bytes (20 CP/M sectors) – loaded to 0x0100

If modifications to the CPMLDR BIOS are required, edit the source files in the cpm3/cpmldr folder and then run “make.sh” (not “make”) to produce CPMLDR.COM.

Follow these steps to install or update CPMLDR.

```
Setup memory mapping.
> map 0 0                     Map a page to ZARC bank 0
> set mmap 1                  Ensure memory mapping is enabled
> bindownload 100             Download to CP/M TPA
$ zarc_dl CPMLDR.COM /dev/ttyUSB0
[reset]
Optional step to disable clear screen on startup.
> write 10e 3e                Change ld de, 0x0225 to ld de, 0x023E
> writesec 2067 20 100        Write 20 MMC sectors (10 KB, 0x2800)
```

6.2.4 “Format” CP/M 2.2 Drive A

No low-level formatting is required as the sectors are already defined on the memory card. However, it is necessary to mark the directory entry blocks as “unused”. The remainder of the drives may be formatted from within CP/M using the format command.

```
Setup memory mapping.
> map 0 0                     Map page 0 to ZARC bank 0
> set mmap 1                  Ensure memory mapping is enabled
> fill 0 4000 e5              Write 0xe5 to every byte
> writesec 4096 32 0          Write to memory card
```

The values above assume 512 directory entities (DPB.DRM is 511). If this value were changed, the directory size would be different. The required size may be calculated:

Number of entries (extents) = DPB.DRM + 1 = 512

Size in bytes: 512 x 32 = 16384 (0x4000)

Number of memory card sectors = 16384 / 512 = 32

6.3 Initial File Transfer

At this point, the CP/M 2.2 system can be started:

```
> cpm2
ZARC CP/M V2.2 BIOS V1.6

A>
```

If the system is newly installed, there will be no commands or utilities available beyond those built into the CCP. It is possible to transfer files using the monitor:

```
> tpadownload                 Start binary download to TPA
$ zarc_dl <file> /dev/ttyUSB0
[reset]
> cpm2                        Start CP/M
ZARC CP/M V2.2 BIOS V1.6
A>save <size> <CP/M filename>
```

<size> is the number of 256 byte pages to save.

A reasonable first file to transfer is the ZARC utility STRAN.COM, as this allows easier and more reliable file transfers. STRAN is available from https://github.com/rundel-tech/ZARC/

6.4 Formatting the Remaining Disks

The ZARC utility FORMAT is recommended for this. It formats a "drive" on the MMC card ready for use by the filing system. Unlike formatters for physical disks, the sectors are already prepared so most of the work is done. All that is required is to mark the directory entries as unused. Format is available from https://github.com/rundel-tech/ZARC/

For use with CP/M 3, consider the following commands when CP/M 3 is running:

```
initdir a:                    Initialise directory for time stamps
set {d:} [create=on,access=off,update=on]    Time stamps on create
and update
set {d:} [name={NAME}]        Set drive label
```

These commands control behaviour for each drive separately. They are persistent, so there is no need to add them to PROFILE.SUB.

```
show {d:} [label]             Show timestamp etc. settings
```

6.5 Installing CP/M 3

Installation of CP/M 3 simply involves copying the system files CPM3.SYS and CCP.COM plus any utilities required to the CP/M 3 A: drive (which is the CPM 2.2 P: drive). CPMLDR will load these files into memory and start the system.

6.6 Building CPM3.SYS

My original download of CPM 3 was from:

http://www.retroarchive.org/maslin/disks/cpm30/index.htm

I found another, claiming to be patched from:

http://www.cpm.z80.de/download/cpm3bin.zip

The description of the latter reads “CP/M 3 BINARY w/YEAR 2000 fixes. : 125K CP/M 3 binary compiled from the developers source. Includes the year 2000 fixes.”

Running “strings BNKBDOS3.SPR” gives:

Maslin ***cpm_z80***
COPYRIGHT (C) 1982,COPYRIGHT 1998, CALDERA, INC. DIGITAL RESEARCH 151282 101198

I chose the later cpm_z80 (1998) version, filling in missing parts where required from the Maslin one.

The required files are stored in cpm3/bios. My approach was to use the sample BIOS code described in the CP/M 3 System Guide section 4 (http://www.bitsavers.org/pdf/digitalResearch/cpm_plus/CPM_Plus_System_Guide_Jan83.pdf). Note that this copy includes the release notes, which some copies elsewhere don’t. http://www.bitsavers.org/pdf/digitalResearch/cpm_plus/ also contains other useful DRI manuals.

The to_cpm folder contains files to be transferred to the CP/M 2.2 system so the BIOS can be built. This is a somewhat messy process. My modules (*.z80) are written in Z80 mnemonics and assembled with zmac on the PC. Note that the link process must be performed under CP/M as the required options are not supported by ld80 (on the PC). In any case, the build utility GENCPM must be run under CP/M.

The following table shows the various files required to build CP/M 3. Files required on the CP/M system are in the to_cpm folder.

File	Source	Description
bioskrnl.z80 (produces BIOSKRNL.REL)	Me, based on DRI's BIOSKRNL.ASM	Kernel module.
scb.z80	DRI, unmodified (except for the name)	Define locations in the System Control Block (SCB).
boot.z80 (produces BOOT.REL)	Me, based on DRI's BOOT.ASM	"Performs system initialization other than character and disk I/O. BOOT loads the CCP for cold starts and reloads it for warm starts."
move.z80 (produces MOVE.REL)	Me, based on DRI's MOVE.ASM	"Performs memory-to-memory moves and bank selects."
chario.z80 (produces CHARIO.REL)	Me, based on DRI's CHARIO.ASM and my CP/M 2.2 BIOS.	"Performs all character device initialization, input, output, and status polling. CHARIO contains the character device characteristics table."
drvtbl.z80	Me, based on DRI's DRVTBL.ASM	"Points to the data structures for each configured disk drive. The drive table determines which physical disk unit is associated with which logical drive."
drvmc.z80 (produces DRVMC.REL)	Me, loosely based on DRI's FD1797SD example module and inheriting code from my CP/M 2.2 BIOS.	"Initialize disk controllers and execute READ and WRITE code for disk controllers. You must provide an XDPH for each supported unit, and a separate disk I/O module for each controller in the system."
time.z80	Me	Time-related code, including real time clock and tick interrupts.
extens.z80	Me	ZARC extensions
BNKBDOS3.SPR *[2]	DRI, unmodified from cpm_z80 site	Banked BDOS component
RESBDOS3.SPR *[2]	DRI, unmodified from cpm_z80 site	Banked BDOS component
CCP.COM *[2]	DRI, unmodified from cpm_z80 site	Console Command Processor

*.z80 files will produce a corresponding *.REL file that must be transferred to the CP/M system for linking with the other modules.

Tools required are:

File	Source	Description
RMAC.COM *$_1$	DRI, from Maslin site	Digital Research Relocatable Macro Assembler
LINK.COM	DRI, from Maslin site	Digital Research Linker LINK-80
GENCPM.COM *$_2$	DRI, from cpm_z80 site	CP/M 3 system generation utility

*$_1$ RMAC.COM is not currently required, but is retained for future use.

*$_2$ Updated on 6/12/2025 to files from John Elliot's 22/1/2025 build at https://www.seasip.info/Cpm/software/dri.html

Files as taken from the cpm_z80 site by preference as this is a later (patched) version. Some files (e.g. RMAC) come from the Maslin site as they are not available on cpm_z80.

I allocated CP/M 2.2 drive E: to build CP/M 3, though any drive with sufficient space could be used.

Running make in the cpm3/bios directory will update the required .REL files using zmac. It also performs a link. The output of this link is not directly usable but the associated .sym file (test_bios.sym) gives the relative addresses of the modules in the BIOS.

Transfer the required tools along with the relevant .REL files to drive E: The script stran.sh transfers any updated files in preparation for the CP/M part of the operations. The following command may be useful to identify recently changed .rel files (within the last hour in this case).

```
find . -name "*.rel" -mmin -60 -ls
```

Build commands (for CP/M 2.2):

```
E>LINK
BNKBIOS3[b,q]=BIOSKRNL,SCB,BOOT,CHARIO,MOVE,DRVTBL,DRVMC,TIME,EXTE
NS,CONTRAN,BTIME                    Note: all one line!
E>GENCPM AUTO                       Omit "AUTO" to change options
E>a:pip p:=cpm3.sys
E>a:pip p:=ccp.com                  Only required for first install
```

Build commands (for CP/M 3) are as above except for the pip commands, which are as follows:

```
E>pip a:=cpm3.sys
E>pip a:=ccp.com                    Only required for first install
```

To run the new image (boot CP/M 3):

```
E>a:zarc exit                       Return to monitor
> cpm3
```

The build process doesn't change CCP.COM, so we only need to copy this once.

From CP/M 3 Command Reference Manual, the link options are:

B - BIOS link in banked CP/M Plus system. Aligns data segment on page boundary. Puts length of code segment in header. Defaults to SPR file type.

Q - Lists symbols with leading question mark.

6.6.1 GENCPM Memory Segment Table

I found an issue with GENCPM's allocation of space for the directory hash table. Allocating space manually fixed this.

We need four bytes per directory entry and we have eight drives. Hence 4 * 512 * 8 = 16384 bytes in total. I put this in bank 2, creating space by specifying the memory segment table as follows:

```
Base,size,bank (00,80,00) 01,86,00
Base,size,bank (00,C0,02) 0D,73,02
Base,size,bank (00,C0,03) 01,BF,03
```

This allocates 0x0d00 to 0x8000 in bank 2 as usable by CP/M. We therefore have the required 4000 bytes in bank 2 starting at 8000. The resulting mapping is:

```
CP/M 3 Sys    8700H 3900H  Bank 00
Memseg No. 00 0100H 8600H  Bank 00
Memseg No. 01 0D00H 7300H  Bank 02
Memseg No. 02 0100H BF00H  Bank 03

CP/M 3 boot map:

BNKBIOS3 SPR  EE00  0E00
BNKBIOS3 SPR  A900  1700
RESBDOS3 SPR  E800  0600
BNKBDOS3 SPR  7B00  2E00

58K TPA
```

7 DEBUGGING

For debugging CP/M applications you can either use the monitor's debugging functions a CP/M debugger such as DDTZ27.COM. I chose DDTZ as it can handle Z80 instructions, though it does use 8080 instruction format with "invented" mnemonics for the Z80-only instructions. Both have the advantages and disadvantages. The monitor's debugger has the advantage that it does not appear in the memory space of the programme being debugged, being mapped in only when required (normally as the result of a trace NMI). As such, it may be used to debug any part of the CP/M system including the BDOS and BIOS. However, for application debugging it is sometimes convenient to use a conventional debugger.

7.1 DDTZ (DDTZ27.COM)

Download from ddtz.zip, manual: DDTZ-DOC.txt

There are two lists of commands at the end of the manual. The version I have responds to the DDTZM commands. The DDTZ manual assumes knowledge of DRI's DDT, so the DDT manual (DDT.pdf) is also useful even though the command names differ in some cases.

Notes:

1) To run a programme that requires command line operands, edit the buffer at 0x80 to insert the command tail. 0x80 itself contains the length. The tail is null terminated.

2) To run to an address, use the command "g,<address>".

3) Trace is followed by an optional count (*not* an address).

4) Use the "e<address>" command to modify memory. ASCII values may be entered if enclosed with single quotes. Enter "." to exit.

7.2 Monitor's Debugger

The monitor's debugger has some advantages. The NMI-based trace allows the complete CP/M environment to be debugged, including the BDOS, BIOS and even monitor calls such a read or write sector. One strategy for debugging CP/M applications (FOO.COM in this example) is:

```
A>FOO.COM                          Do not press return
[Press NMI button]
>t 100                             Start of CP/M programme
[Press <return>]
```

Some time later, the monitor should be entered with PC = 0100. This method requires a few seconds as the command load operations are traced, including disk reads.